

Helios: Protocol, Attacks, Repair and Scoped Ballot Secrecy

Formal reference to the Lean development

KC Sivaramakrishnan
FP Launchpad, IIT Madras

16 September 2026

Abstract

This reference explains the historical homomorphic Helios protocol, the replay and permutation attacks, component weeding, and the Lean proof of symbolic ballot secrecy for the documented repair. It connects the mathematical objects to their public declarations and organizes the proof by semantic dependency. Applied-pi-calculus infrastructure and symbolic homomorphic computation are treated as verified project-local libraries through their interfaces. Their internal proofs are outside the exposition. A concrete probabilistic attack, repair, finite DDH reduction and parameterized secrecy theorem are also checked. The computational endpoint retains explicit attacker resource conditions and external implementation/efficiency arguments. The models have separately documented scopes; no symbolic-to-computational correspondence or general transfer theorem has been proved.

Contents

1 Election, ballots and public transcript	3
2 Verified symbolic library interfaces	4
3 Validation, component weeding and the tuple correction	4
4 Running example: attacks and failed repairs	5
5 The security contract	6
6 Symbolic ballot secrecy	8
7 Proof architecture	9
8 Counterexamples to proof shortcuts	10
9 Concrete ballot proofs	11
10 Honest ballots and nonce collisions	12
11 The public probabilistic attack	13
12 Concrete repair and honest correctness	14
13 Computational game and ballot secrecy	15
14 Comparing the two scoped Helios models	22

15 Why there is a separate development catalogue	24
16 Scope and limitations	24
A Exact computational Lean interface	26
B Concept-to-source index	28

Reading this reference

The protocol and symbolic observation model come first, followed by symbolic secrecy, concrete cryptographic algorithms, the probabilistic attack and repair, and computational secrecy. The two results have different scopes: the symbolic repair and observation schedule are not identified with the computational game. Section 14 lists the differences. For a computational reading, start with Sections 9–13 (algorithms, attack, repair and secrecy), then read Sections 14 and 16 for the scope comparison and limitations. Sections 1–8 develop the symbolic model.

Mathematical displays are transcriptions or explicitly labelled compressions of linked Lean declarations. Symbolic candidate indices are one-based in the text; Lean uses zero-based finite indices. $=_E$ denotes equality in the symbolic equational theory, while ordinary $=$ denotes mathematical equality. The protocol provenance is Cortier–Smyth’s historical homomorphic Helios description, especially §5 and Figure 4 [1], with the tuple correction stated below.

Lean checks the proofs under the cited definitions. The axiom audit accepts `propext`, `Classical.choice`, and `Quot.sound`; it does not certify the prose transcription, external efficiency arguments, or fidelity to deployed software. The separate [historical development catalogue](#) records supporting component constructions from source revision 9273cdc. It is supplementary to the mathematical argument here.

1 Election, ballots and public transcript

There are $\ell = n + 1 > 0$ candidates, two honest voters, a bulletin board, a trustee and $q \in \mathbb{N}$ additional voter inputs. The honest voters submit first, on restricted channels. The board publishes their ballots in order. Subsequent inputs use separate public voter channels and are checked against all preceding accepted ballots. The board and trustee follow the specified programs. The theorem concerns one such finite election for every q .

Let k be the secret key, $K = \text{pk}(k)$ the public key, and r_{ij} the fresh nonce for honest voter $i \in \{0, 1\}$ and candidate j . A vote is a vector of ground symbolic terms $v = (v_1, \dots, v_\ell)$ satisfying

$$\text{CandidateValues}(v) \iff \sum_{j=1}^{\ell} v_j =_E 0 \vee \sum_{j=1}^{\ell} v_j =_E 1.$$

The sum is a nonempty fold of symbolic addition. This condition entails that each component is E -equal to zero or one. It permits arbitrary equivalent term representatives, including the all-zero abstention vector.

Lean: `CandidateValues`.

Lean: `CandidateSubstitution.bit_representative`.

Definition 1.1 (Honest ballot; notation-level transcription). *For voter i with vector v , define*

$$\begin{aligned} c_{ij} &= \text{penc}(K, r_{ij}, v_j), & p_{ij} &= \text{spk}(K, r_{ij}, v_j, c_{ij}), \\ C_i &= \prod_{j=1}^{\ell} c_{ij}, & R_i &= \bigcirc_{j=1}^{\ell} r_{ij}, \\ p_i^\Sigma &= \text{spk}\left(K, R_i, \sum_j v_j, C_i\right). \end{aligned}$$

The ballot is the tuple

$$B_i(v) = (c_{i1}, \dots, c_{i\ell}, p_{i1}, \dots, p_{i\ell}, p_i^\Sigma).$$

Products and nonce compositions are nonempty folds. Proof terms have four arguments, including the ciphertext statement.

Lean: `Historical.General.ballot`.

Lean: `Historical.General.aggregateProof`.

For each candidate, the board multiplies the two honest ciphertext fields and the corresponding fields of every subsequently accepted ballot. It sends the tuple $T = (T_1, \dots, T_\ell)$ privately to the trustee. The reply contains $D_j = \text{partialDecrypt}(k, T_j)$. The board publishes the tuple D , then the ordered result tuple with entries $\text{dec}(D_j, T_j)$.

Lean: `boardTally`.

Lean: `trusteeBody`.

Lean: `resultBody`.

The source does not insert a separate public broadcast of T or of each additional ballot. Those inputs remain visible attacker interactions. Publishing individual partial decryptions would change the aggregate-only transcript specified here.

Lean: `boardFinish`.

2 Verified symbolic library interfaces

The underlying libraries are project-local components verified using Lean and Mathlib. This section states the cryptographic interface used by Helios; normalization algorithms, binding implementations and generic proofs are left to the source index. Here *source* means the actual applied-pi-style protocol program and its operational states, as opposed to the auxiliary election phase model used in the proof. A *frame presentation* is a derivation using the calculus’s structural rules that puts a state’s exported substitutions into a ground frame with explicit name restrictions; it is evidence connecting the program state to the frame used for observation tests.

Equations used by the protocol. $\text{penc}(K, r, m)$ represents encryption, $\text{spk}(K, r, m, c)$ a proof term, $\text{checkspk}(K, c, p)$ its public check, and $\perp$ a tuple terminator. The generating theory contains:

$$\begin{aligned} \text{fst}(\text{pair}(x, y)) &=_E x, & \text{snd}(\text{pair}(x, y)) &=_E y, \\ 0 + 1 &=_E 1, & 0 + 0 &=_E 0. \\ \\ \text{dec}(k, \text{penc}(\text{pk}(k), r, m)) &=_E m, \\ \text{dec}(\text{partialDecrypt}(k, \text{penc}(\text{pk}(k), r, m)), \text{penc}(\text{pk}(k), r, m)) &=_E m, \\ \text{penc}(K, r, m) \cdot \text{penc}(K, s, u) &=_E \text{penc}(K, r \circ s, m + u), \\ \text{checkspk}(K, \text{penc}(K, r, b), \text{spk}(K, r, b, \text{penc}(K, r, b))) &=_E \text{ok} \quad (b \in \{0, 1\}). \end{aligned}$$

Multiplication, addition and nonce composition are associative and commutative. E is the congruence closure of these equations. The interface adds no general unit laws, no ciphertext identity, and no rule extracting the arguments of an encryption or proof. The symbols denote this theory, not the full laws of a concrete cryptographic group.

Lean: [Equation](#).

Lean: [EqE](#).

Recipes. A frame φ assigns a ground term to each currently exported handle. A recipe is any finite term over those handles and public names, using the whole term signature. Evaluating R substitutes the frame’s values, written $R\varphi$. Publicness $\text{Public}_H(R)$ means that no literal name in R lies in the current restricted-name set H . Publicness checks all four proof arguments. Handles may nevertheless refer to ciphertexts containing restricted names. This is how an attacker manipulates an opaque received value.

Lean: [Term.Public](#).

Lean: [Frame.eval](#).

Operational interface. The applied-pi-style library supplies restricted names and channels, parallel composition, local term bindings, input, output and conditionals. Its interfaces distinguish structural rearrangement, internal reduction, public input and fresh-handle output. Helios uses these original source actions. The proof derives the connection to an election phase model; it does not take that connection as an axiom. Section 5 states the observation and action contract without reproducing the calculus’s foundational rules.

3 Validation, component weeding and the tuple correction

Write $\pi_j(b) = \text{fst}(\text{snd}^{j-1}(b))$ and $C(b) = \prod_{j=1}^{\ell} \pi_j(b)$. There are $m = 2\ell + 1$ fields.

Definition 3.1 (Repaired acceptance). *For public key K , earlier ballot list L , and submitted term b :*

$$\begin{aligned}
\text{ProofValid}(K, b) &\iff \text{checkspk}(K, C(b), \pi_{2\ell+1}(b)) =_E \text{ok} \\
&\quad \wedge \forall j \in \{1, \dots, \ell\}, \text{checkspk}(K, \pi_j(b), \pi_{\ell+j}(b)) =_E \text{ok}, \\
\text{TailGuard}(b) &\iff \text{snd}^m(b) =_E \perp, \\
\text{NoReuse}(L, b) &\iff \forall a \in L \forall i, j \in \{1, \dots, \ell\}, \neg(\pi_i(a) =_E \pi_j(b)), \\
\text{Accepted}(K, L, b) &\iff \text{ProofValid}(K, b) \wedge \text{TailGuard}(b) \wedge \text{NoReuse}(L, b).
\end{aligned}$$

The fixed candidate-count parameter $n = \ell - 1$ is suppressed here.

Lean: `Accepted`.

Lean: `ProofValid`.

Lean: `NoReuse`.

Lean: `TailGuard`.

The source program implements this finite formula. Its agreement theorem holds for every ground environment, including malformed inputs. Each additional input is followed by the guard. Success continues collection; failure selects the null continuation, so this execution never reaches tally publication. The first two honest ballots are relayed without this guard, as in Figure 4.

Lean: `electionGuard_holds`.

Lean: `collectBallots`.

Counterexample 3.2 (Printed next-field check). *Tuples are right-nested pairs terminated by $\perp$. For a canonical m -field tuple, $\pi_{m+1}(b) =_E \text{fst}(\perp)$, and $\text{fst}(\perp) \neq_E \perp$. The printed next-field guard therefore rejects a canonical tuple. A prefix of m fields followed by `pair( $\perp, t$ )` instead passes that guard for arbitrary t .*

Lean: `canonical_tuple_fails_printed_guard`.

Lean: `extra_bottom_passes_printed_guard`.

The correction checks `sndm(b)`. A canonical one-candidate ballot then passes on an empty board; it cannot be accepted on a board already containing it. These controls alone are not a privacy theorem. The correction preserves E and is not a claimed published erratum.

Lean: `oneCandidate_corrected_accepts`.

Lean: `accepted_excludes_replay`.

4 Running example: attacks and failed repairs

Use two candidates X and Y and three voters Alice, Bob and Mallory. In world 0, Alice selects X, encoded (1,0), and Bob selects Y, encoded (0,1); world 1 exchanges those choices. Both honest tallies are (1,1). Mallory sees the honest ballots before submitting under the third voter identity.

Example 4.1 (Replay and permutation). *Copying Alice’s ballot adds her vote a second time. Exchanging its two ciphertext/proof positions adds the opposite candidate instead. The aggregate certificate remains valid under permutation because its statement is the same commutative product.*

Submission	Acceptance policy	World 0	World 1
Fresh X ballot	Any of the three	(2, 1)	(2, 1)
Alice's ballot	Original	(2, 1)	(1, 2)
Alice's ballot	Whole-ballot check	Reject	Reject
Permuted Alice	Whole-ballot check	(1, 2)	(2, 1)
Permuted Alice	Component weeding	Reject	Reject
Fresh abstention	Component weeding	(1, 1)	(1, 1)

These totals follow by adding the vote vectors. Lean's finite-model fixtures check them independently of the informal calculation. The original policy validates proofs; the whole-ballot policy additionally rejects an identical previous payload; component weeding checks reuse across every candidate position.

Lean: [ExplainableCrypto.Helios.replay_tallies](#).

Lean: [ExplainableCrypto.Helios.permutation_tallies](#).

Lean: [ExplainableCrypto.Helios.ordinary_tallies](#).

Lean: [ExplainableCrypto.Helios.fresh_abstention_tallied](#).

Theorem 4.2 (Finite-model distinguishers). *For replay under the original policy, the public test “is the X count 2?” returns true in world 0 and false in world 1. For permutation under the whole-ballot policy, “is the X count 1?” has the same separating behavior. The finite model’s universal recipe/view equality property therefore fails for both policies.*

Lean: [ExplainableCrypto.Helios.replay_distinguishes](#).

Lean: [ExplainableCrypto.Helios.original_not_private](#).

Lean: [ExplainableCrypto.Helios.permutation_distinguishes](#).

Lean: [ExplainableCrypto.Helios.wholeBallot_not_private](#).

Ciphertexts in this model are allocated handles, and proofs are ideal certificates. The adversary has a finite recipe language. Its view contains the honest board, submitted payload and either a rejection reason or a tally. The full partial-decryption transcript is absent. These theorems certify the attack mechanism at that scope; the final secrecy theorem uses richer semantics.

Lean: [ExplainableCrypto.Helios.RestrictedPrivacy](#).

Fresh ballots being accepted and tallies changing with fresh votes exclude universal rejection and constant-output explanations of the controls. Blocking the two attacks motivates the repair; general secrecy still needs Sections 5–7.

Lean: [ExplainableCrypto.Helios.fresh_ballots_accepted](#).

Lean: [ExplainableCrypto.Helios.tally_not_constant](#).

5 The security contract

5.1 All public observations

Definition 5.1 (Frame static equivalence). *For frames φ, ψ with the same complete handle domain and the same restricted-name policy H :*

$$\varphi \approx_s \psi \iff \forall R, S, \text{Public}_H(R) \wedge \text{Public}_H(S) \implies (R\varphi =_E S\varphi \iff R\psi =_E S\psi).$$

Recipes range over the full finite-term language, with no fixed size bound.

Lean: `Frame.StaticEq`.

This tests equality after arbitrary public computation, including projection, homomorphic combination, checking proofs and decryption attempts. Failed checks remain terms and may participate in equality tests. Frame values need not be pointwise equal across the swapped elections.

For an actual source state a , a frame presentation is an original structural derivation from its frame projection to a restricted ground substitution. Source static equivalence requires presentations for both states under common policies and static equivalence of their ground frames. Presentation is evidence, not an assumed interpretation of arbitrary syntax.

Lean: `Named.RepresentsFrame`.

Lean: `Named.StaticEq`.

The complete public frame grows as follows. A handle names one entire tuple where the payload is a tuple; the attacker can project it.

Handle count	Exported values in canonical order
1	Public key
2	Public key, first honest ballot
3	Public key, both honest ballots
4	Previous values, aggregate partial-decryption tuple
5	Previous values, ordered result tuple

During extra inputs and their checks, the frame has three handles. A rejected run retains that frame and cannot complete the board's publication sequence. Input labels still record the attacker's channel and recipe.

Lean: `sourceView`.

Lean: `sourceFinalFrame`.

5.2 Matching behavior as well as frames

Write $a \xrightarrow{\tau} a'$ for an internal source reduction and $\Rightarrow$ for its finite reflexive-transitive closure. $a \xrightarrow{l} a'$ is an original free action with label l . A fresh output on channel c has target domain $\text{Option}(\text{Fin}(h))$: `some( $x$ )` retains an old handle and `none` is the new handle. Both worlds enumerate it with the same bijection $\iota = \text{outputHandle}$ to $\text{Fin}(h + 1)$.

Definition 5.2 (Source weak labelled bisimulation; compressed interface). *A family $\mathcal{R}_h$ on source states with h handles is a weak labelled bisimulation when it is symmetric and, for every $\mathcal{R}_h(a, b)$:*

1. *Both states are closed and satisfy source static equivalence.*
2. *If $a \xrightarrow{\tau} a'$, there is b' with $b \Rightarrow b'$ and $\mathcal{R}_h(a', b')$.*
3. *If l is scoped in a and $a \xrightarrow{l} a'$, there is b' with $b \Rightarrow \xrightarrow{l} b'$ and $\mathcal{R}_h(a', b')$.*
4. *If a has a fresh output on c to a' , then b matches that same channel with a fresh output surrounded by internal closures to b' , and $\mathcal{R}_{h+1}(a'[\iota], b'[\iota])$.*

States are weakly labelled bisimilar, written $a \approx_\ell b$, when some such family relates them. Symmetry supplies the converse action obligations.

Lean: `Named.IsWeakLabelledBisimulation`.

Lean: `Named.WeakLabelledBisimilar`.

The existential union is itself a bisimulation. This is the checked largest-relation formulation on the documented finite, static-channel source syntax. The definition includes both free action forms; reachable historical states separately exclude old-handle free outputs.

Lean: `Named.weakLabelledBisimilar_isWeakLabelledBisimulation`.

Lean: `SourceElectionRelation.no_handle_output`.

Static equivalence alone cannot establish this contract. Two states can have identical current observations while one can publish and the other is stopped; Section 8 gives a checked instance. The theorem asserts this weak bisimilarity, not an additional fairness or eventual-completion property.

6 Symbolic ballot secrecy

Let L, R be valid ground candidate substitutions. In world 0 the honest voters use (L, R) ; in world 1 they use (R, L) . Nonces stay attached to voter and candidate positions. Define

$$\mathcal{E}_s = \text{scopedVoterElection}(ns, s, L, R, q, ch), \quad s \in \{0, 1\}.$$

This is the actual scoped source election: each voter allocates a candidate nonce, binds its ciphertext and proof, and continues to the next candidate. The administration restricts its key/auxiliary names and private channels and initially exports the public key. A normalization theorem connects that program to the convenient election body used in the proof.

Lean: `scopedVoterElection`.

Lean: `scopedVoterElection_normalizes`.

Exact parameter conditions. The name record contains $k = ns.secretKey$, an auxiliary name a , and $r_{ij} = ns.nonce(i, j)$. Its freshness condition is

$$k \neq a \wedge (i, j) \mapsto r_{ij} \text{ is injective} \wedge \forall i, j, r_{ij} \neq k \wedge r_{ij} \neq a.$$

For each of $v = L.value$ and $v = R.value$, also require

$$\text{NoncesFreshFor}(ns, v) \iff \forall i, j, t, r_{ij} \notin \text{nameSupport}(v_t).$$

This checks the full syntax of vote representatives, before reduction modulo E . The channel record has a broadcast channel, a trustee channel and a voter channel for every natural-number index. Its voter map is injective; the broadcast and trustee channels differ from each other and every voter channel. The first two voter channels and trustee channel are restricted by the operational policy. Distinct channel numbers alone do not imply privacy.

Lean: `Historical.Names.Fresh`.

Lean: `NoncesFreshFor`.

Lean: `Channels.Fresh`.

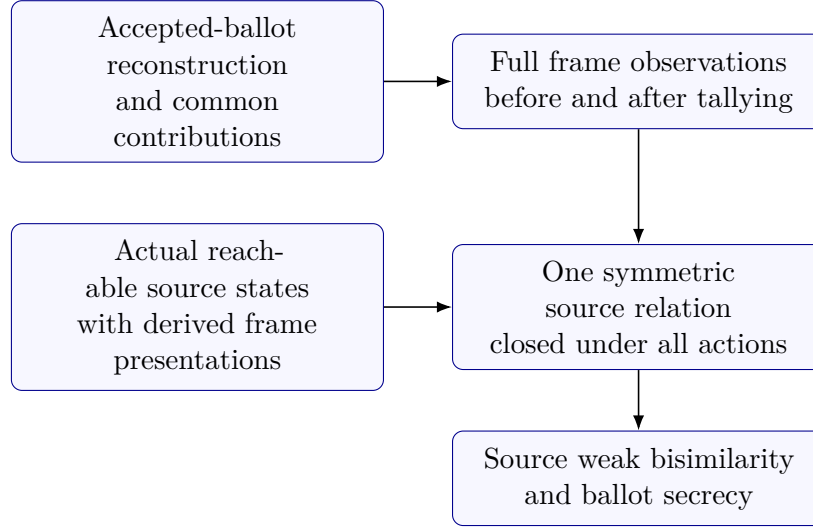
Theorem 6.1 (Historical symbolic ballot secrecy). *Fix any $n \in \mathbb{N}$ and $q \in \mathbb{N}$. Let ns be a name record satisfying the freshness condition above, and let ch be a fresh channel record. Let L and R be valid ground candidate substitutions, with $\text{NoncesFreshFor}(ns, L.value)$ and $\text{NoncesFreshFor}(ns, R.value)$. Then the repaired historical elections satisfy*

$$\mathcal{E}_0 \approx_\ell \mathcal{E}_1.$$

Lean: `scopedVoterElection_ballot_secretcy`.

Validity proofs are fields of the candidate-substitution type. Abstention and arbitrary E -equivalent ground representatives are included. There is no caller-supplied static-equivalence, normalization, frame-presentation or action-correspondence premise. The result retains the documented repair, full E observations and rejection behavior. It makes no computational encryption or zero-knowledge claim.

7 Proof architecture



The diagram separates cryptographic observations from operational correspondence. Both are required by the final theorem.

7.1 What an accepted ballot contributes

Proof validity yields semantic witnesses for the bit-valued ciphertext fields and both kinds of proof. This is initially an existence result about terms; it does not claim the attacker can recover honest secret nonces or votes. The stronger accepted-ballot reconstruction uses publicness, the corrected tail and the presence of both honest ballots on the board to reconstruct a public constructor recipe for the submitted ballot.

Lean: `proofValid_iff_values`.

Lean: `Historical.General.accepted_ballot_constructor_recipe`.

Here is the cross-world consequence used for accepted contributions. Write φ_s for the three-handle honest frame. Under name freshness, valid L, R , a public recipe r under $ns.restricted$, and a recipe board containing both honest ballot handles, acceptance of $r\varphi_0$ supplies one valid literal bit vector b and public nonce recipes ρ_j :

$$\forall s \in \{0, 1\} \forall j, \quad \pi_j(r\varphi_s) =_E \text{penc}(K, \rho_j\varphi_s, b_j).$$

This compresses the `SharedBallotData` record and its `Explains` predicate. Shared objects are the bits and recipes; evaluated nonce values need not be equal across worlds. Applying the result along accepted sequences preserves the contribution alignment.

Lean: `Historical.General.initial_accepted_common_data`.

Lean: `Historical.General.accepted_sequence_common_data`.

7.2 Acceptance and all transcript observations

For public key, board and submission recipes, static equivalence transports acceptance and rejection, also along board extensions. The honest-frame theorem supplies the initial static-equivalence premise.

Lean: `Historical.General.initial_accepted_iff`.

Lean: `Frame.StaticEq.acceptsSequence_iff`.

Under $ns.Fresh$, valid L, R , and world-0 phase reachability for the chosen finite administration size, the completed observation result is

$$\text{sourceView}(ns, 0, L, R, p) \approx_s \text{sourceView}(ns, 1, L, R, p).$$

This covers all public recipe equalities on the actual partial-decryption and result expressions, including their tuple projections.

Lean: `reachable_source_view_staticEq`.

7.3 From phases to arbitrary reachable source states

Phases describe collection, checking and publication. Connecting them to arbitrary structurally rearranged source states requires three further facts.

First, every finite original execution from $\mathcal{E}_s$ yields a reached phase, name/channel and complete-handle bijections, a semantic phase connection, and a structural presentation of the full frame. The inputs are the execution and the final theorem’s freshness conditions.

Lean: `source_reachable_frame_presentation`.

Second, `SourceElectionRelation` carries both actual execution histories, one common reached phase and coordinate system, semantic openings of both states and both complete frame presentations. Initial membership is derived from the scoped voters. Presentations are explicit because semantic realization equality alone is insufficient (§8).

Lean: `SourceElectionRelation`.

Lean: `SourceElectionRelation.initial`.

Third, actual partner actions preserve this same relation:

Action	Preserved obligation
Internal	Match communication or either guard outcome in the actual partner.
Public input	Keep the literal channel and recipe and both successor presentations.
Fresh output	Keep the channel, every old handle, the fresh handle and full payload; derive the needed target name policy.
Structural change	Retain actual execution provenance and frame evidence.
Handle renaming	Use a common bijection preserving the complete exported domain.

Lean: `SourceElectionRelation.internal`.

Lean: `SourceElectionRelation.free`.

Lean: `SourceElectionRelation.bound`.

Lean: `SourceElectionRelation.structural`.

Lean: `SourceElectionRelation.reindex`.

The final assembly packages symmetry, closedness, observations and successor clauses as a weak labelled bisimulation. Matching uses one partner internal step and zero surrounding silent steps for visible actions. Instantiation at the actual initial source states proves the theorem in §6.

Lean: `source_reachable_weak_bisimulation`.

8 Counterexamples to proof shortcuts

These controls explain why some tempting simplifications would lose the target. They are retained evidence; none is a missing premise in the final theorem.

Counterexample 8.1 (Semantic equality does not give structural presentation). *Consider the source-local equation $\nu x.\{x/x\}$ and the empty process. Under the realization semantics they have the same inhabited class: the self-reference permits any local value. The original structural rules cannot erase that self-reference to obtain the empty process.*

Lean: [SourceStructuralOpeningSPOT.semantic_equality_is_insufficient](#).

Lean: [SourceStructuralOpeningSPOT.insufficient_semantic_pair_is_inhabited](#).

Adding that unconstrained local to a complete live frame also fails to supply a ground frame presentation, even though the semantic opening to the unpadding state exists. This is a false general reconstruction claim, not a failed tactic. The proof instead derives presentations for actual reachable election states, using the necessary structural evidence.

Lean: [SourceBoundRigiditySPOT.changing_frame_values_or_policy_cannot_present_cycle](#).

Counterexample 8.2 (An old frame policy need not describe the next output). *The process $\nu a.\bar{c}(a).0$ has an empty old frame, which admits an empty restricted-name policy. After fresh-handle publication, the frame records $x \mapsto a$ under the restriction of a . The old empty policy cannot present the resulting frame. An appropriate new policy can.*

Lean: [SourceOutputFramePresentationSPOT.private_output_reconstructs_its_policy](#).

Lean: [SourceOutputFramePresentationSPOT.reconstructed_policy_cannot_be_the_old_empty_policy](#).

The fixture uses base name 40 and public channel 0. This is a failure of an insufficient policy invariant. Fresh-output reconstruction derives the target policy from the whole live process, rather than freezing it to the old frame’s minimal policy.

Counterexample 8.3 (Static equivalence alone misses behavior). *An output-capable state and a stopped state have equivalent current frames but are not weakly labelled bisimilar. The stopped state cannot silently manufacture the output channel. A separate control rules out declaring a missing exported handle domain equivalent to a complete one.*

Lean: [SourceBallotSecrecySPOT.static_equivalence_does_not_prove_weak_bisimilarity](#).

Lean: [SourceBallotSecrecySPOT.missing_domain_is_not_weakly_bisimilar](#).

At the historical source level, a replay input and subsequent rejection are matched by the final relation. The theorem is also instantiated for every finite additional-input count. These complement the finite model’s independent valid-ballot and nonconstant-tally controls.

Lean: [SourceBallotSecrecySPOT.actual_replay_rejection_is_weakly_matched](#).

Lean: [SourceBallotSecrecySPOT.arbitrary_administration_secrecy](#).

9 Concrete ballot proofs

The [computational development](#) uses VCVio for probability semantics and cryptographic algorithms, with Lean/Mathlib 4.33.1 for the shared formal development. The exact vulnerable construction is Smyth’s at-most-one variant with abstention and component weeding [2]. The [concrete model](#) records its public transcript and construction boundaries. The computational secrecy statement is in Section 13.

Write ElGamal as $E_h(r, m) = (g^r, g^m h^r)$ in a prime-order group. Each branch $i \in \{0, 1\}$ of a ballot proof contains (A_i, B_i, c_i, s_i) and verifies

$$g^{s_i} = A_i a^{c_i}, \quad h^{s_i} = B_i (b/g^i)^{c_i}, \quad c_0 + c_1 = H(A_0, B_0, A_1, B_1).$$

The last equation is shared by both branches. The hash input here contains only commitments. Honest zero/one proof constructors satisfy these equations for every hash function; the algebraic lemmas allow arbitrary scalar values. The encryption body is identified with VCVio’s algorithm. A separate historical sampler assigns probability $1/(|F| - 1)$ to each nonzero scalar. With injective generator exponentiation, honest encryption never produces $(1, 1)$.

Lean: `proveZero_valid`.

Lean: `proveOne_valid`.

Lean: `sampleNonzero_probability`.

Lean: `honestEncrypt_ne_neutral`.

Theorem 9.1 (Neutral-ciphertext proof). *For any scalars e, z, w , set*

$$\begin{aligned} (A_0, B_0) &= (g^w, h^w), & (A_1, B_1) &= (g^z, h^z g^e), \\ c_1 &= e, & c_0 &= H(A_0, B_0, A_1, B_1) - e, \end{aligned}$$

and $s_0 = w, s_1 = z$. This proof verifies for $(a, b) = (1, 1)$ for every H , without a secret key.

Lean: `neutralProof_valid`.

Replacing an earlier valid ballot’s components by their product in the first slot and $(1, 1)$ in all other slots preserves proof validity: copy its aggregate proof into the first and aggregate proof positions, and use neutral proofs elsewhere. For at least two components, this ballot passes weeding exactly when its aggregate and $(1, 1)$ each avoid every earlier board component. This characterizes freshness on arbitrary boards; Section 10 gives a sufficient nonce condition and a probability bound for honest ballots.

Lean: `proofReuse_valid`.

Lean: `proofReuse_fresh_iff`.

Independent fixtures use $p = 23, q = 11, g = 2, h = 8$ and a fixed hash challenge 5. Lean checks the constructors’ encoded group values and the verification equations; changing a response is rejected. These fixtures test arithmetic, not cryptographic hardness or SHA-256. Section 11 connects the attack to probability. The concrete repair is checked in Section 12; Section 13 states secrecy under explicit implementation assumptions.

Lean: `Controls.neutral_constructor_matches`.

Lean: `Controls.changed_response_rejected`.

10 Honest ballots and nonce collisions

The checked two-component honest ballot for $v \in \{0, 1\}$ is $(E_h(r_0, v), E_h(r_1, 0))$. Its component and aggregate proofs use the concrete zero/one constructors; its aggregate encrypts v with nonce $r_0 + r_1$. All proofs validate for arbitrary proof coins and every commitment hash. This covers abstention and the first candidate, as needed for the intended swapped-world attack experiment.

Lean: `honestBallot_valid`.

Let the second honest ballot use nonces s_0, s_1 . Suppose all four nonces are nonzero and generator exponentiation is injective. Define

$$\text{Good}(r, s) \iff (\forall i, j \in \{0, 1\}, r_i \neq s_j) \wedge (\forall j \in \{0, 1\}, r_0 + r_1 \neq s_j).$$

Under this condition, the second honest ballot passes component weeding against the first, and the proof-reuse ballot from the first passes weeding against both. This holds for either honest vote and

arbitrary proof coins. The conditions are derived from equality of actual ciphertext components. They do not assume a symbolic correspondence.

Lean: `honestBallot_fresh`.

Lean: `proofReuse_honest_fresh`.

For a finite field F and fixed r_0, r_1 , draw s_0, s_1 independently and uniformly from $F \setminus \{0\}$. Then

$$\Pr[\exists j \in \{0, 1\}, s_j \in \{r_0, r_1, r_0 + r_1\}] \leq \frac{6}{|F| - 1}.$$

Lean derives this union bound from VCVio's probability semantics and the nonzero sampler. Repeated forbidden values only loosen the bound.

Lean: `pairCollision_probability_le`.

Retained counterexample. If the second nonces agree, both honest ballots contain the same zero component. Their proofs validate, but the second ballot fails weeding. Thus unconditional acceptance of independently sampled honest ballots is false. The counterexample is a general checked implication, not a failed search for an acceptance proof.

Lean: `coincident_honest_ballots_not_fresh`.

The full honest sampler extends this bound through both nonce pairs and all proof randomness. It does not resample on collisions or discard rejected runs.

Lean: `drawHonestPair_collision_le`.

11 The public probabilistic attack

The implemented experiment has two candidates, two honest voters, one dishonest voter and one honest trustee. The challenger samples a uniform hidden bit v ; honest ballots encode v and $1 - v$ in the first candidate. Trustee secrets, proof coins and encryption nonces use the historical nonzero sampler.

The attacker sees the parameters, key proof, fingerprint, honest decisions and accepted identified ballots. It submits the first ballot's aggregate followed by a neutral ciphertext, with the reused and neutral proofs above. Submission checks are actual proof verification and component weeding. Rejection retains the board and tallying continues. The final public message includes the submission decision, accepted board, encrypted tallies, decryption shares and proofs, and bounded tally decoding (including decoding failure).

Lean: `submit_rejected_preserves_board`.

Lean: `executeAttack_key_proof_valid`.

Lean: `executeAttack_decryption_proofs_valid`.

On `Good`, all three ballots are accepted and public decryption of the first tally yields g^{1+v} . The attacker returns true exactly when this group element equals g^2 . It uses neither the secret key nor honest coins.

Lean: `executeAttack_on_good`.

Theorem 11.1 (Concrete proof-reuse success). *Let F be a finite field and G an F -module with decidable equality. Suppose $r \mapsto r \bullet g$ is injective. For every choice of the typed ballot, key-proof, decryption-proof and fingerprint hashes, the public proof-reuse attacker in `ballotSecrecyGame` satisfies*

$$\Pr[\text{guess} = v] \geq \max\left(0, 1 - \frac{6}{|F| - 1}\right).$$

The Lean statement uses truncated subtraction in ENNReal. The proof bounds all bad-event executions; freshness is derived, not assumed. The bound exceeds one half when $|F| - 1 > 12$.

Lean: `instantiated_attack_game`.

Lean: `ballotSecrecyGame_proofReuse_success`.

Independent modulo-23 fixtures check the published trustee proofs, decoded tallies 1 and 2, invalid-proof rejection, retained-board tallying after honest rejection, and a fresh abstention with no copying effect.

Lean: `ExecutionControls.rejection_retains_board_and_tally_continues`.

Lean: `ExecutionControls.fresh_abstention_has_no_copying_effect`.

This is a cast-only attack witness with typed subgroup inputs and a fixed eligible-identity schedule. Byte parsing, non-subgroup inputs, an authentication implementation and a security-parameter/PPT backend are outside the checked game. The optional audit-and-discard path is unused. The concrete repair follows next; Section 13 gives its scoped secrecy theorem.

12 Concrete repair and honest correctness

The selected repair follows Bernhard–Pereira–Warinschi [4]: hash the statement as well as commitments, and include implicit aggregate ciphertexts in weeding. For a ballot proof, the challenge now hashes $(g, h, (a, b))$ and the four commitments. Key and decryption proofs also hash their statements. The branch equations and homomorphic encryption remain the same.

Lean: `strongHonestBallot_valid`.

For ballot b , let $C(b)$ contain its components and implicit aggregate. Expanded weeding requires

$$\forall b' \in \text{board}, \forall c \in C(b), \forall c' \in C(b'), \quad c \neq c'.$$

A submitted first component equal to an earlier aggregate fails this test. This is a check on the actual group ciphertexts, independent of hash security.

Lean: `copied_aggregate_not_expanded_fresh`.

Theorem 12.1 (Known attack rejected). *In the repaired three-voter execution, Smyth’s copied-aggregate submission is rejected as a reused ciphertext for every choice of honest coins and typed hashes. The previously accepted board is retained and tallying continues. Under the finite-field sampler, the rejection probability is one.*

Lean: `executeRepairedAttack_rejected`.

Lean: `repairedSubmit_rejected_preserves_board`.

Lean: `repairedAttackWorld_rejection_probability`.

Honest correctness and collisions. Honest strong proofs validate for every typed hash. If the two honest nonce sets $\{r_0, r_1, r_0 + r_1\}$ and $\{s_0, s_1, s_0 + s_1\}$ are disjoint and generator exponentiation is injective, both ballots are accepted. Their decrypted group tally is $(g, 1_G)$ in either swapped world. For independently sampled nonzero coins, accidental rejection has probability at most $9/(|F| - 1)$. The extra three terms cover the second aggregate nonce; no collision runs are removed.

Lean: `repairedHonestPair_decrypts`.

Lean: `repairedHonestPair_rejection_le`.

Both repair components matter. With weak hashes, rerandomizing (a, b) to (ag^u, bh^u) and changing each response to $s_i + c_i u$ preserves verification for every hash. Conversely, strong statement hashes alone still validate Smyth’s copied aggregate: its ciphertext statement is unchanged. Independent controls retain both failures and check accepted honest ballots, literal trustee transcripts and nonconstant tallies under the full repair.

Lean: `Proof01.rerandomize_valid`.

Lean: `strongProofReuse_valid`.

Lean: `RepairControls.honest_valid_attack_rejected_and_tally_preserved`.

The implemented instance retains the historical nonzero sampler and cast-only scope. BPW uses full-field sampling and a different simulated-tally privacy game. Its published theorem is not an imported reduction. Adaptive extraction for the actual disjunctive proof and the finite DDH reduction are checked; Section 13 gives the final theorem with external efficiency.

13 Computational game and ballot secrecy

Fix a security parameter n , prime scalar field $F_n = \mathbb{Z}/q(n)\mathbb{Z}$ and group G_n with distinguished element g_n . We use additive group notation, so the public key is $x \bullet g_n$. The notation dictionary from Section 9 is

$$g^r \longleftrightarrow r \bullet g_n, \quad uv \longleftrightarrow u + v, \quad 1_G \longleftrightarrow 0_G.$$

Thus the same ElGamal ciphertext $(g^r, g^m h^r)$ is written $(r \bullet g_n, m \bullet g_n + r \bullet h)$, with $h = x \bullet g_n$; only the group notation changes. The election has two candidates, honest voters 0 and 1, attacker-controlled voter 2, and one honest trustee. The secret bit selects which honest voter casts the first-candidate vote and which abstains; the second candidate coordinate is always zero. This section uses the statement-bound hashes and component/aggregate freshness repair of Section 12.

Lean: `ElectionOracle.preparedGame`.

Lean: `ElectionOracle.game`.

Lean: `ElectionOracle.world`.

Lean: `ElectionOracle.prefixWithCoins`.

Lean: `ElectionOracle.finishWithCoins`.

Lean: `ElectionOracle.run_ask`.

Definition 13.1 (Ballot-secrecy bias). *For the experiment in Figure 1, define*

$$\beta_A(n) = \left| \Pr[\text{BS}_A(n) = 1] - \frac{1}{2} \right|.$$

The probability includes preparation, the secret bit, protocol and attacker randomness, and the shared random oracle. A nonnegative function f is negligible when $n^d f(n) \rightarrow 0$ for every fixed $d \in \mathbb{N}$.

Lean: `ElectionSecurityFamily.Family.bias`.

The game exposes complete retained-board and rejection behavior. It does not assert privacy for other observation schedules, corrupt trustees, more voters, or a larger challenge family. The conditions below describe exactly the encoded attacker interface covered by the theorem.

13.1 Parameters and attacker resources

Write $F_n = \mathbb{Z}/q(n)\mathbb{Z}$. A value A of `ElectionSecurityFamily.Family` supplies the following data and properties. All polynomials have natural coefficients. The resource conditions concern the *original* callbacks, before the derived query clocks are applied.

Guessing experiment $\text{BS}_A(n)$

1. Start with an empty shared random-oracle cache H . Run $i \leftarrow A_{\text{prepare}}(n)$; its queries update H .
2. Draw a uniform bit b . Draw the trustee secret, key-proof nonce, two decryption-proof nonces and both honest ballots' coins using the specified nonzero samplers. No collision is conditioned away.
3. Construct the key proof and public parameters. Construct and submit honest ballots for choices b and $\neg b$, in voter order, using repaired validation. Let v contain the parameters, fingerprint, both submission decisions and retained honest board.
4. Run $(B, s) \leftarrow A_{\text{cast}}(n, i, v)$. Validate B as voter 2 against the retained board. Rejecting B preserves that board; either decision proceeds to tallying.
5. Aggregate the retained board, produce both trustee shares and their proofs, and perform bounded decoding. Form w from v , submitted ballot B , the decision, final board, encrypted tally, shares, proofs and optional decoded results, including decoding failure.
6. Run $\hat{b} \leftarrow A_{\text{guess}}(n, i, s, w)$. Return $[\hat{b} = b]$.

All proof generation, validation and attacker callbacks share H . Hash keys are tagged by proof type and contain the full statement and commitment. A repeated key reuses its answer; a new key receives a uniform field element. Attacker callbacks also have the specified finite-uniform sampling interface.

Figure 1: The fixed computational ballot-secrecy game. The cache and private states are not added to the public transcript; the attacker retains its own query answers. This is a compression of the original game, before query clocking.

Data or condition	Required property
Scalar and group family	For each n , $q(n)$ is prime; G_n is an additive commutative group and an F_n -module, with decidable equality.
Group representation	An injective encoding of every G_n element into bits, with one polynomial upper bound $R(n)$ on its length.
Scalar width	The binary width of $q(n)$ is at most a polynomial $Q(n)$. A public scalar is encoded by the bits of its canonical representative.
Fingerprint	Its output has binary width at most a polynomial $F(n)$ for every parameter record whose candidate and eligible lists are exactly $[0, 1]$ and $[0, 1, 2]$. All typed group/scalar assignments to such records are covered.
Generator	The map $r \mapsto r \bullet A.\text{generator}(n)$ is injective.
Private state	Arbitrary families Init and Saved , each with an injective bit encoding. Private sizes below are actual encoding lengths.
Original source	The preparation computation and the preparation-dependent cast/guess callbacks, with the existing typed oracle interface.
Preparation	At most $P(n)$ queries; every output in its source support has encoded Init length at most $I(n)$.
Casting	At most $C(n + i + \text{encPrefix}(v))$ queries for every initial state i and public prefix v . Every cast output has encoded saved state length at most $S(n + i + \text{encPrefix}(v))$.

Guessing	At most $J(n + i + s + \text{encResult}(w))$ queries for every initial state i , saved state s , and public result w .
----------	--

Here P, I, C, S, J are the fields `prepareCost`, `initGrowth`, `castCost`, `savedGrowth`, and `guessCost`, respectively. Query bounds and source support range over *all well-typed replies*, including inconsistent answers to repeated hash queries. Pure output growth is bounded separately: a query count does not bound local work or private storage. These are resource properties, not a supplied correspondence with the game.

Lean: [ElectionSecurityFamily.Family](#).

Derived representation and coverage. The public lengths above are literal encodings of every field of the existing `PublicPrefix` and `PublicResult`. Generic injectivity preserves observations; specialization to the established prime codecs is exact. The checked serialization bounds include all nested framing. In particular, let $L = R + Q + F + 1$, $B_0 = 100(L + 1)$ and $B_1 = B_0 + 1000(L + 1)$. Reached structural widths are bounded by $B_0(n)$ and $B_1(n)$, and reached encoded lengths by $10^8 B_0(n)$ and $10^{10} B_1(n)$. The fixed constants are conservative proof bounds, not measured costs. The source proof retains rejected submissions: there are at most two board entries before casting and three afterward, and any successful decoded value is at most three.

Lean: [ElectionPublicEncoding.prefix_injective](#).

Lean: [ElectionPublicEncoding.result_injective](#).

Lean: [ElectionPublicEncoding.prime_prefix_eq](#).

Lean: [ElectionPublicEncoding.prime_result_eq](#).

Lean: [ElectionPublicEncoding.prefix_length_le](#).

Lean: [ElectionPublicEncoding.result_length_le](#).

Polynomial composition therefore gives

$$\begin{aligned} V &= X + I + 10^8 B_0, & C_* &= C \circ V, & S_* &= S \circ V, \\ Z &= X + I + S_* + 10^{10} B_1, & J_* &= J \circ Z. \end{aligned}$$

The normalized family $D = A.\text{prepared}$ keeps preparation unchanged and clocks casting and guessing at $C_*(n)$ and $J_*(n)$. Its saved type is `Option Saved`. A cast timeout returns an explicit zero ballot and absent state; a guess timeout or absent saved state returns false. No private default or acceptance of this fallback is assumed. The reached-input bounds prove clock transparency, including the original private states and public results. Thus `coverage` is equality of complete oracle computations; `run_coverage` retains the full output and final shared cache from every starting cache. `bias_eq` identifies the original game's bias.

Lean: [QueryClock.eq_of_total_bound](#).

Lean: [ElectionSecurityFamily.Family.coverage](#).

Lean: [ElectionSecurityFamily.Family.run_coverage](#).

Lean: [ElectionSecurityFamily.Family.bias_eq](#).

13.2 Accuracy and the secrecy theorem

The main DDH reduction uses repeated extraction attempts. For a fixed *accuracy degree* $k \in \mathbb{N}$, its accuracy request is

$$a_k(n) = (n + 1)^{k+1}.$$

The finite reduction has residual slack $2/a_k(n)$ plus terms that are negligible under the stated assumptions. For each fixed k , the number of extraction attempts is polynomial in n ; increasing

k changes that polynomial and may change the realizing machine. The assumption is $\forall k \exists M_k, P_k$, not a single bound jointly polynomial in n and varying k . The inequality $n^k 2/a_k(n) \leq 2/(n+1)$ explains why a separate reduction for every fixed accuracy degree suffices to prove negligibility. One fixed inverse-polynomial slack would not suffice.

Lean: [ElectionSecrecyPrototype.accuracy](#).

Lean: [ElectionSecrecyPrototype.negligible_of_polynomial_accuracy](#).

Write $\text{Eff}_{\text{enc}}(D)$ when one finite uniform fair-coin machine realizes the exact output distribution of the family D and halts within a fixed polynomial in n on every encoded challenge and every coin path. Its precise machine convention and relation to ordinary PPT are given below. Let $D_{A,k}^{\text{main}}$ and D_A^{reject} denote the exact main and rejection distinguishers constructed from the normalized family $A.\text{prepared}$. For a distinguisher D , define its DDH advantage by

$$\Delta_D(n) = |\Pr[D_n(g_n, xg_n, yg_n, (xy)g_n) = 1] - \Pr[D_n(g_n, xg_n, yg_n, zg_n) = 1]|, \quad x, y, z \leftarrow F_n.$$

Scalar multiplication abbreviates $x \bullet g_n$, and each challenge uses the same fixed injective group encoding as the election family.

Lean: [ElectionSecrecyFairBits.main](#).

Lean: [ElectionSecrecyFairBits.reject](#).

Lean: [ReductionEfficiency.Efficient](#).

Lean: [ReductionEfficiency.DDHassumption](#).

Theorem 13.2 (Computational ballot secrecy; compressed mathematical statement). *For every parameter and attacker family A satisfying all conditions in Section 13.1, let $D = A.\text{prepared}$ be the normalization defined above. Suppose:*

1. $1/(q(n) - 1)$ is negligible;
2. for every fixed k , $\text{Eff}_{\text{enc}}(D_{A,k}^{\text{main}})$;
3. $\text{Eff}_{\text{enc}}(D_A^{\text{reject}})$;
4. for every distinguisher family E with $\text{Eff}_{\text{enc}}(E)$, its DDH advantage Δ_E is negligible.

Then the original game's ballot-secrecy bias is negligible:

$$\forall d \in \mathbb{N}, \quad \lim_{n \rightarrow \infty} n^d |\Pr[\text{BS}_A(n) = 1] - \frac{1}{2}| = 0.$$

Lean: [ElectionSecurityFamily.Family.ballot_secrecy](#).

Hypotheses 2 and 3 are the usual proof step that the constructed reductions run in polynomial time. They are explicit here because the development defines a machine-cost model but does not prove polynomial-time realizations of these two complete reductions. Section 13.4 gives the external efficiency arguments and relates the chosen machine convention to conventional PPT.

The first assumption is the Lean quantity `noncePointBound` for the prime field, namely $1/(q(n) - 1)$. It is equivalent here to negligible $1/q(n)$: for primes $q \geq 2$, $1/q \leq 1/(q - 1) \leq 2/q$. The theorem retains both explicit implementation hypotheses; it does not derive them from query counts. The full Lean interface appears in Appendix A.

13.3 How the computational reduction proves secrecy

The checked reduction retains preparation, both public interaction points, rejection decisions and the public transcript fields. It first replaces the honest proofs with programmed random-oracle simulations, accounting for the difference introduced by the historical nonzero sampling rule. Repeated extraction obtains witnesses for accepted attacker contributions except on a bounded failure event, while retaining the original execution and saved state. An extracted finisher produces the public

result without access to the DDH challenge’s secret scalar. A separate distinguisher accounts for observable honest rejection.

In the mathematical random-DDH comparison hybrid, which has a secret-key argument, a uniform mask can be translated by the honest vote without changing its distribution. The complete unscored public continuation is thereby independent of the secret bit, so the guess succeeds with probability $1/2$. This is a claim about the full continuation, not just its tally. The extraction and simulation comparisons connect this hybrid to the actual game.

Lean: [ElectionDDHSource.completed_random_win](#).

Lean: [ElectionSecrecyPrototype.prepared_negligible_of_ddh](#).

For the normalized family, write $p(n) = P(n)$ and $c(n) = C_*(n)$. For every fixed k , the checked ideal-sampling reduction gives, for all sufficiently large n ,

$$\beta_A(n) \leq \frac{18}{q(n) - 1} + \frac{3(11p(n) + 2c(n) + 131)}{q(n)} + \frac{2}{a_k(n)} + \Delta_{\text{main},k}^{\text{ideal}}(n) + \Delta_{\text{reject}}^{\text{ideal}}(n).$$

Here the two ideal advantages use the exact finite-uniform samplers before fair-bit replacement. The field-size side condition of the finite bound holds eventually because p, c, a_k are polynomial and the inverse field size is negligible. Fair-bit transport adds negligible sampling losses and replaces these two advantages by those of the exact machine-realized reductions. DDH makes the latter negligible; varying the fixed accuracy degree as above removes the remaining inverse-polynomial slack. Finally, the checked equality of the clocked and original games identifies their biases.

Lean: [ElectionSecrecyPrototype.accuracy_admissible](#).

Lean: [ElectionSecrecyFairBits.prepared_negligible_of_fair_bits](#).

Lean: [ElectionSecurityFamily.Family.bias_eq](#).

Correspondence with the four Lean hypotheses

Beyond A and its family conditions, `Family.ballot_secrecy` has exactly these four hypotheses and concludes negligible original prepared-game bias.

Hypothesis	Meaning
<code>hsize</code>	The original <code>noncePointBound</code> of F_n is negligible. This lower-growth requirement on field size is separate from encoded-width upper bounds.
<code>hmain</code>	<code>MainReductionEfficient</code> for D and the same group representation: for every fixed accuracy degree k , one uniform finite program realizes the exact normalized main reduction within a polynomial in n .
<code>hreject</code>	<code>RejectionReductionEfficient</code> for that exact D and representation.
<code>hddh</code>	DDH against the explicitly defined uniform machine class under the same representation and generator family.

A realization contains finite coin-only code, a polynomial clock, halting with an actual Boolean output on every coin path, and exact output-distribution agreement on every challenge quadruple. Its input is unary n followed by four separately framed group words. Before a coin command the answer tape must be blank, excluding an uncharged bulk erasure. There are no arithmetic, callback, hash or storage oracles in this machine. Each fixed k may choose its own code and polynomial; no

bound jointly polynomial in varying k is asserted. The constant-false control inhabits the generic realization predicate, not the exact reductions or the combined cryptographic hypotheses.

Lean: `ReductionEfficiency.Realization`.

Lean: `ReductionEfficiency.MainReductionEfficient`.

Lean: `ReductionEfficiency.RejectionReductionEfficient`.

Lean: `ElectionSecurityFamily.Family.ballot_secretcy`.

13.4 Relation to conventional PPT and standard DDH

External mathematical argument. Every family satisfying Eff_{enc} has an implementation by a conventional strict probabilistic polynomial-time Turing machine on the same encoded challenges. Only this inclusion is needed; no equivalence of machine classes is claimed as a Lean theorem.

To see the inclusion, fix a realization. Its finite labels and three head cells, each holding a Boolean or blank, determine a finite transition table. A local command performs only a fixed number of head moves or cell writes. Hash commands are excluded. On every valid run, the answer tape is blank before a fair-coin command, so that command can be implemented by a coin flip and one local write, without bulk erasure. Thus a conventional multitape probabilistic machine simulates the steps with bounded overhead; a single-tape simulation has polynomial overhead. This is a simulation of the defined tape operations, not a claim about the runtime of evaluating the Lean interpreter.

The realization guarantees halting within a polynomial $T(n)$ only on valid challenge encodings. To obtain a strict machine on arbitrary bit strings, parse the unary parameter and field framing, reject malformed framing, and enforce the known polynomial clock while simulating. No group-membership test is needed: a string with well-formed framing but invalid group words can time out and return a fixed bit. Parsing costs polynomial time in input length N , and $n \leq N$. Clock construction and simulation therefore cost polynomial time in N on all inputs. On valid challenges the clock does not alter the output distribution. The checked bound $N \leq n + 8R(n) + 5$, where R bounds group encoding width, also makes this runtime polynomial in the security parameter on those challenges.

Lean: `ReductionEfficiency.inputWord`.

Lean: `ReductionEfficiency.inputWord_length`.

Lean: `ReductionEfficiency.Realization`.

Consequently, *standard DDH against conventional strict PPT machines for these same encodings and challenge distributions implies the DDH hypothesis used by the Lean theorem*. Challenges lie in the order- $q(n)$ subgroup generated by g_n ; the formal parameter conditions do not require g_n to generate all of G_n . A DDH assumption stated with a different representation would additionally need an efficient translation preserving the distribution. Injectivity and polynomial encoded width alone do not supply that translation. This external inclusion establishes neither of the exact reduction-efficiency hypotheses; those are the separate implementation claims discussed next.

External claim E1: attacker membership. Use uniform, explicitly polynomial-clocked strict oracle-PPT callback implementations with effective canonical representations. Charge writing query indices, processing replies, copying suspended configurations and serializing returned private state. A strict time polynomial in unary n plus actual encoded input length bounds both query count and serialized private output. Enlarging constants and coefficients gives P, I, C, S, J . Hash replies have the stated scalar width. A uniform-range index must be written by the attacker; its reply has at most that index's polynomial bit width. Effective scalar/group operations and fingerprint evaluation obey the fixed parameter policy. These facts supply local family resources; the

Lean coverage proof, rather than an external game-correspondence hypothesis, derives transparency of the query clocks on reached inputs.

The clock convention applies to every typed reply sequence. A guarantee only against consistent hash functions is not the same premise. The independent finite control asks the same key twice and makes a third request only on unequal answers: every fixed singleton-key handler gives two requests, but its complete tree needs three. This refutes transfer of the same finite bound, not a general asymptotic claim. Given a known strict polynomial bound on all consistent executions, an external time-clock normal form preserves their oracle interactions, private outputs and output law, and stops other reply paths. That normalization is an external argument; `Family.coverage` does not itself establish it for an arbitrary pre-existing unlocked source callback. The stated membership claim uses the explicitly clocked model. Expected-time-only attackers are outside this argument.

Lean: `ElectionCoverageControls.fixed_hash_two_requests`.

Lean: `ElectionCoverageControls.consistent_bound_not_syntactic_bound`.

External claim E2: exact normalized reduction efficiency. Fix k and uniform correct implementations of the actual primitives and callbacks, including all altered replay response paths. Their local runtime, full configuration widths, scalar/group widths, query indices and stored responses have polynomial envelopes. The actual normalized budgets are $p = P(n)$ and $c = C_*(n)$, evaluations of fixed polynomials; thus the repetition count is uniformly computable, not merely bounded by a computable function. The checked query bounds give

$$M = 4P(n) + 4C_*(n) + 78, \quad R_k = 3456(P(n) + C_*(n) + 10)^3(n + 1)^{4(k+1)}.$$

Rejection uses at most M queries and main at most $(1 + 3R_k)M + 4 + 4J_*(n)$. The rejection prefix includes the actual cast.

Lean: `ElectionDDHSource.honestReject_total_bound`.

Lean: `ElectionDDHSource.ballotSecrecy_prime_cost`.

Implement dictionaries and logs as finite lists. The representation invariant identifies auxiliary, live and programmed lookups with their source caches, retains the sticky flag and ordered history, and identifies callback snapshots and replay tapes with their original configurations and answers. A hit returns the stored value without sampling; a miss performs the source draw and update. Programming follows the actual occupied-key rule and keeps the live cache separate. Induction over these operations preserves the invariant. The checked finite-cache laws and auxiliary-source interpretation identify the operations; they do not by themselves compile this whole implementation.

Lean: `runBallotFiniteCache_eq`.

Lean: `runBallotFiniteCache_length_le`.

Lean: `ElectionReplaySource.lower_run`.

Each replay attempt starts from the same recorded original path. Copying a snapshot or reconstructing it with recorded answers has polynomial cost. A failed residual's enlarged state is not passed to the next attempt. The final continuation uses the original retained state/cache. Source bad, rejection, extraction-failure and normalized-timeout branches remain executed. Using the same fresh fair bits for corresponding draws preserves the exact output law, not only a support condition or a successful-run projection.

Lean: `ballotJointReplayRepeatAtPath`.

Lean: `ElectionDDHSource.prefix_repeated_original`.

Lean: `ElectionDDHSource.extractedGame`.

The bounded sampler reads the source-prescribed number of bits and computes modulo the requested range, with polynomial binary arithmetic. Each sampled bit can be copied into work storage and the native coin cell cleared before the next draw. This enforces the blank-answer invariant with constant overhead per bit. No additional approximation is needed: the existing fair-bit theorem already accounts for the source sampling loss.

For a common polynomial width envelope $W(n)$, an elementary-operation bound $b(W + n)$, and $U = M + J_*(n) + 1$, conservative handwritten time bounds are

$$\begin{aligned} T_{\text{reject}} &= O(T_{\text{prep}} + T_{\text{cast}} + M^2 b(W + n)), \\ T_{\text{main},k} &= O((1 + 3R_k)(T_{\text{prep}} + T_{\text{cast}} + U^2 b(W + n)) \\ &\quad + T_{\text{guess}} + U^2 b(W + n)). \end{aligned}$$

These charge storage scans, copying, original-state reconstruction, parameter setup, primitive computation, counters and final guessing. Coefficients may depend on the fixed k . They are external asymptotic arguments, not checked instruction counts or inferred local runtimes from query counts. Finite three-tape coin-only code has the ordinary polynomial-time interpretation under the stated step convention; this mathematical model argument is not a Lean standard-PPT adequacy theorem.

Trust boundary. The following table complements every Family condition and all four theorem hypotheses above. Readers assess the models and the external arguments.

Layer	Evidence and limit
Kernel and libraries	Lean checks the encoded proofs and standard axiom dependencies under pinned definitions. This audit does not validate E1/E2; neither is a new global Lean axiom.
Models and assumptions	Fixed protocol, encodings and machine convention; prime/module family, injectivity, widths, field-size negligibility and same-representation DDH remain explicit.
External E1	Uniform strict-PPT implementation implies the local Family resources under the all-typed-reply clock convention described above.
External E2	Correct uniform primitive/callback implementations and the cache/replay argument give exact normalized-reduction realization and polynomial runtime for each fixed accuracy degree.

14 Comparing the two scoped Helios models

The results concern two separately scoped models of the same historical protocol. No correspondence theorem, computational soundness theorem or refinement between these models is proved. The table distinguishes *aligned* features from choices that need *reconciliation* and *potential obstructions* to identifying the present models. A potential obstruction is a concrete mismatch, not an impossibility result for every future abstraction.

Dimension	Symbolic model	Computational model and assessment
Size and choices	$n + 1$ candidates, two honest voters and arbitrary finite extra inputs; valid bit vectors include abstention.	Fixed two candidates and three voters; swap first-candidate vote with abstention. Reconcile election and challenge families.

Dimension	Symbolic model	Computational model and assessment
Repair	Component and aggregate proof checks, corrected tuple tail, cross-position component weeding.	Statement-bound proof hashes and component/aggregate freshness. Potential obstruction: different acceptance predicates.
Cryptography	Historical E1–E9 and AC on terms; no identity/inverse or public proof-response operations.	Full group/scalar algorithms, identity and zero nonce sums, explicit commitments and responses. Potential obstruction: richer concrete operations and equalities.
Attackers	Unbounded public recipes and actual source actions within finite process scope; explicit handle/private-name policies.	Preparation, cast and guess oracle computations with encoded input-sensitive query/growth conditions. Reconcile operation language, resources and external strict-PPT interpretation.
Trustee	One honest secret; private tally exchange; published partial/result tuples.	One honest secret, key proof and aggregate decryption shares/proofs. Aligned trust setting; reconcile messages. Neither covers adaptive trustee corruption.
Board and schedule	Honest inputs relayed in order; later scheduled ballots checked against earlier entries.	Honest board, fixed eligible identities, two honest attempts and one attacker submission. Aligned sequential board; reconcile size and validation scope.
Rejection	Failed guard leads to the null process, with no subsequent tally publication.	Explicit decision and unchanged board; computation continues to the retained-board tally. Potential obstruction: observably different failure behavior.
Public prefix	Key handle and both honest ballot outputs.	Complete parameters, fingerprint, key proof, decisions and honest board in a prefix record. Reconcile extra fields and observation timing.
Public result	After successful collection, partial tuple then result tuple; no extra attacker-ballot or tally-ciphertext broadcast.	Submitted ballot, decision, full accepted board, encrypted tally, shares/proofs and optional decoded tally. Potential obstruction to direct trace equality.
Tally	Nonempty homomorphic products; symbolic partial decryption yields bounded numerals.	Candidatewise sums, group shares and bounded decoding with failure represented by an option. Aligned aggregation; reconcile decoding and rejection.
Freshness	Restricted fresh names with explicit candidate/nonce/channel hypotheses.	Nonzero random sampling with collisions retained, plus statistical/replay/fair-bit losses. Reconcile probabilistic interpretation; distinct names are not automatically random samples.
Secrecy	Weak labelled bisimilarity, including all frame tests and matching internal/free/bound actions.	Negligible guessing bias under all Family conditions, DDH, field-size and exact reduction-efficiency hypotheses. Reconcile notions; neither theorem transports the other.

The critical definitions can be inspected independently of status prose:

Lean: `Accepted`.

Lean: `NoReuse`.

Lean: `collectBallots`.

Lean: `boardFinish`.

Lean: `sourceView`.

Lean: `scopedVoterElection_ballot_secretcy`.

Lean: `Ballot.ExpandedFreshFor`.

Lean: `ElectionOracle.prefixWithCoins`.

Lean: `ElectionOracle.finishWithCoins`.

The `full scope comparison` supplies source anchors for every row. A future common specification must choose or relate the repair, rejection and publication policies before using these as two interpretations of one protocol. Instantiating the symbolic candidate/voter counts does not align those policies. The internal computational clocking equalities relate two computational games, not computational records to symbolic frames. A correspondence theorem would need to resolve these differences explicitly.

15 Why there is a separate development catalogue

The symbolic proof depends on reconstruction of reachable source states as well as equality of public observations. The computational proof uses simulation, extraction and a finite DDH bound before deriving negligible bias. Its two exact reduction-efficiency hypotheses remain external implementation claims.

The `historical catalogue` preserves checked arithmetic, encoding, cache and partial machine-execution constructions. Those results do not supply a complete implementation of either reduction. They document an attempt to mechanize runtime more concretely than the theorem requires under its stated hypotheses. Component code volume and commit timestamps do not measure the inherent difficulty of a cryptographic proof or establish a limitation of Lean or its libraries.

The `experience report` records measured source snapshots and the precise pinned-library interfaces examined. An actual Turing-machine backend exists; the examined composition and iteration interfaces did not themselves deliver the required complete reduction witness. This is a finding about those interfaces and this construction attempt, not an impossibility theorem. The useful methodological question is which implementation and resource arguments a security statement leaves to its reader.

16 Scope and limitations

Concrete attack and repair. The symbolic equations omit a ciphertext identity. Membership in $\mathbb{Z}_p^*$, the concrete condition in the historical discussion, does not by itself exclude coordinates $(1, 1)$. Smyth’s separate replay analysis identifies an identity-operation/proof-reuse issue [2]. The `repair analysis` records that historical boundary. Section 12 implements a subsequent concrete repair; rejecting one attack does not prove computational security. The tuple-tail correction is a separate issue.

No cross-model transfer. The two secrecy results concern separately scoped models. A common election semantics would need to reconcile the acceptance, rejection, observations and cryptographic operations listed in Section 14. Cheval et al.’s ProVerif framework addresses privacy and

verifiability but explicitly excludes homomorphic tallying [3]; it supplies no existing adapter for this case. Verifiability and receipt-freeness require their own definitions and arguments.

What is reused today. Mathlib and pinned VCVio are the root project’s direct external Lake dependencies. Mathlib supplies proof infrastructure and testing/display packages. The applied-pi and homomorphic symbolic libraries were developed locally; the symbolic secrecy endpoint does not import VCVio. Computational modules use VCVio’s probability semantics, algorithms and reductions. Reference checkouts remain independent of Lake’s dependency copies. The completed [reuse audit](#) records adapters and their limits. This PDF does not claim those libraries could be replaced without new correspondence work.

A Exact computational Lean interface

The following source fragments are copied without changes from `ElectionSecurityFamily.lean`. The document gate compares their bytes with the current declarations before compiling this PDF. Line wrapping and mathematical glyph rendering are typographic only. These parameters apply to the declarations:

```
variable {q : Nat → Nat} [∀ n, Fact (q n).Prime] {G : Nat → Type}
  [∀ n, AddCommGroup (G n)] [∀ n, Module (ZMod (q n)) (G n)]
  [∀ n, DecidableEq (G n)]
```

Here $q\ n$ is prime and $G\ n$ is an additive commutative group with scalar action by $ZMod\ (q\ n)$. Additive notation represents ElGamal group operations; it does not expose a discrete logarithm.

```
structure Family (q : Nat → Nat) (G : Nat → Type)
  [∀ n, Fact (q n).Prime] [∀ n, AddCommGroup (G n)]
  [∀ n, Module (ZMod (q n)) (G n)] [∀ n, DecidableEq (G n)] where
  representation : GroupRepresentation G
  scalarWidth : Polynomial Nat
  scalar_width : ∀ n, (q n).size ≤ scalarWidth.eval n
  fingerprintWidth : Polynomial Nat
  fingerprint : ∀ n, PublicParameters (ZMod (q n)) (G n) → Nat
  fingerprint_width : ∀ n parameters,
    parameters.candidates = [0, 1] → parameters.eligibleVoters = [0, 1, 2] →
      (fingerprint n parameters).size ≤ fingerprintWidth.eval n
  generator : ∀ n, G n
  generator_injective : ∀ n,
    Function.Injective (fun r : ZMod (q n) => r • generator n)
  Init : Nat → Type
  Saved : Nat → Type
  encodeInit : ∀ n, Init n → List Bool
  encodeSaved : ∀ n, Saved n → List Bool
  init_injective : ∀ n, Function.Injective (encodeInit n)
  saved_injective : ∀ n, Function.Injective (encodeSaved n)
  prepare : ∀ n, Comp (ZMod (q n)) (G n) (Init n)
  adversary : ∀ n, Init n → Adversary (ZMod (q n)) (G n) (Saved n)
  prepareCost : Polynomial Nat
  initGrowth : Polynomial Nat
  castCost : Polynomial Nat
  savedGrowth : Polynomial Nat
  guessCost : Polynomial Nat
  prepare_queries : ∀ n, (prepare n).IsTotalQueryBound (prepareCost.eval n)
  prepare_size : ∀ n initial, initial ∈ support (prepare n) →
    (encodeInit n initial).length ≤ initGrowth.eval n
  cast_queries : ∀ n initial before,
    ((adversary n initial).castBallot before).IsTotalQueryBound
    (castCost.eval (n + (encodeInit n initial).length +
      prefixInputWidth representation n before))
  cast_size : ∀ n initial before output,
    output ∈ support ((adversary n initial).castBallot before) →
    (encodeSaved n output.2).length ≤ savedGrowth.eval
    (n + (encodeInit n initial).length +
      prefixInputWidth representation n before)
  guess_queries : ∀ n initial saved view,
    ((adversary n initial).guessVote saved view).IsTotalQueryBound
    (guessCost.eval (n + (encodeInit n initial).length + (encodeSaved n saved).length +
      resultInputWidth representation n view))
```

Lean: [ElectionSecurityFamily.Family](#).

`GroupRepresentation` includes an injective bit encoder with a polynomial width bound for every group element. Scalar encodings are the binary digits of the canonical residue.

`prefixInputWidth` and `resultInputWidth` are lengths of the actual complete public encodings, including fingerprints, proofs, boards, decisions and optional decoded results. Private `Init/Saved` values also have injective encodings. Query counts and private output sizes are bounded separately; a computation making no queries can still produce a large private output. All source reply branches are quantified, including inconsistent replies that a shared random oracle would never issue.

Lean: [ReductionEfficiency.GroupRepresentation](#).

Lean: [ElectionSecurityFamily.prefixInputWidth](#).

Lean: [ElectionSecurityFamily.resultInputWidth](#).

Exact theorem statement

In namespace `ElectionSecurityFamily.Family`, with the source parameter variable `(A : Family q G)`, the complete theorem statement is:

```
theorem ballot_secrecy
  (hsize : negligible (fun n => noncePointBound (ZMod (q n))))
  (hmain : MainReductionEfficient A.prepared A.representation)
  (hreject : RejectionReductionEfficient A.prepared A.representation)
  (hddh : DDHAssumption (q := q) A.representation A.generator) :
  negligible (fun n => ENNReal.ofReal (A.bias n))
```

The proof body is omitted; the statement and every hypothesis are retained.

Lean: [ElectionSecurityFamily.Family.ballot_secrecy](#).

`A.bias n` is the original attacker’s guessing bias, not the bias of a substituted stage model or merely a tally difference. The theorem’s four hypotheses are reciprocal-field-size negligibility, main-reduction efficiency, rejection-reduction efficiency, and DDH. They supplement every condition in `Family`; those conditions do not disappear because they are structure fields. `A.prepared` is the explicitly clock-normalized family derived inside this development. Both efficiency hypotheses refer to its exact reductions. Game equality alone would not transfer runtime to an extractor’s altered replay paths.

B Concept-to-source index

Paths are relative to `ExplainableCrypto/Helios/`; symbolic modules are inside its `Symbolic/` directory. Links open source files; the main text’s anchors identify declarations within them.

Concept	Starting module(s)
Finite attack semantics	Model.lean
Replay and permutation	Replay.lean , Permutation.lean
Independent finite controls	SPOT.lean
Symbolic term interface	Symbolic/Terms.lean , Equations.lean
Vote and ballot construction	CandidateSubstitutions.lean , GeneralCandidateFrames.lean
Repaired validation	Symbolic/Ballot.lean , SourceElectionGuard.lean
Tuple correction	Symbolic/TupleGuard.lean
Board and trustee	SourceElectionSyntax.lean , SourcePayloadSyntax.lean
Actual scoped voters	SourceScopedVoterElection.lean
Accepted contributions	AcceptedSequences.lean
Full public observations	SourceCapturedViews.lean
Reachable reconstruction	SourceReachableFramePresentation.lean
Successor relation	SourceElectionRelation.lean
Final theorem and controls	SourceBallotSecrecy.lean , SourceBallotSecrecySPOT.lean
Concrete ballot proofs	Computational/BallotProof.lean , ProofReuse.lean
Honest nonce collisions	Computational/HonestBallot.lean , CollisionProbability.lean
Public probabilistic attack	Computational/BallotSecrecyGame.lean , AttackGame.lean
Concrete repair	Computational/RepairedExecution.lean , RepairedSampling.lean
Computational game and secrecy	Computational/ElectionOracle.lean , ElectionSecurityFamily.lean
Exact efficiency hypotheses	Computational/ReductionEfficiency.lean
Historical component constructions	Separate development catalogue
Interactive ballot proof	Computational/BallotSigma.lean , NonzeroSimulationBridge.lean
Oracle simulation	Computational/AdaptiveBallotSimulation.lean
Replay extraction	Computational/BallotForkAcceptance.lean
Proof-target provenance	Computational/BallotProofProvenance.lean
Replayable programming	Computational/BallotProgrammedOracle.lean
Honest prefix and decisions	Computational/HonestPrefixOracle.lean
First submission extraction	Computational/RepairedSubmissionSource.lean

Other demonstrations have separate roles: [one-time-pad mutations](#), [finite specification mutations](#), and [testing support](#). These are navigation links, not additional premises of the Helios theorem.

References

- [1] V. Cortier and B. Smyth. *Attacking and fixing Helios: An analysis of ballot secrecy*. [Author preprint](#). Protocol and symbolic model: §5; replay and permutation attacks: §3.
- [2] B. Smyth. *Replay attacks that violate ballot secrecy in Helios*. [May 2012 manuscript](#).
- [3] V. Cheval, V. Cortier, A. Debant and F. Moser. *Simultaneously Proving Privacy and Verifiability: A ProVerif Framework for Internet Voting*. CSF 2026. [Publication page](#).
- [4] D. Bernhard, O. Pereira and B. Warinschi. *How not to Prove Yourself: Pitfalls of the Fiat–Shamir Heuristic and Applications to Helios*. ASIACRYPT 2012; [ePrint version](#).